OK, we should be good to go—let's try this out. With the app still running, navigate to http://localhost:3000/users in a browser. You should see the familiar Rails scaffold for creating and listing users as shown in the following figure. Make sure you can create and delete users.



Great, we have Postgres all set up with Compose.

Decoupling Data from the Container

It's great that we've set up our database so we can persist data in our Rails app. However, currently, there's a major failing in how it works. Let's see what the problem is, and see how to get around it.

Part of the philosophy of using Docker is that we should treat containers as ephemeral—throwaway things that we spin up, use, and then delete. However, our Postgres database is running in a container and persisting our data by writing and modifying files *on disk* inside the container. What happens to our data if we delete our database container? Yep, you've guessed it: we say bye bye to all our lovely data. Not really what we want.

Now that we're going to be storing important data in our database, we need to think a bit more carefully about this.

Just like in our code where we try to decouple things that change frequently from things that don't, we want to decouple our *data* from the containers that generate and use it. Our data should be stored *separately* from the container running the database. That way, we could delete, remove, and recreate the container without affecting the data.

The answer: we store persistent data in volumes, which by their very nature are decoupled from the life cycle of containers. Even if we delete a container with a connected volume, the volume continues to exist independently, storing our data safely. We can then recreate the container, hooking up the volume, and everything is hunky-dory.